INFORME DE LABORATORIO 10 · TECNOLOGÍA WEB I

Laboratorio 10 — TELNET vs SSH: la prueba del espía, la identidad del servidor y el acceso con claves ed25519

Informe de resultados: capturas reales de TELNET y SSH, regeneración de la identidad del servidor, acceso por clave y publicación por scp

Asignatura	Tecnología Web I (SIS-0150) · Sexto semestre · Presencial
Master Class	10 de 15 — TELNET vs SSH
Modalidad	Individual
Criterio de verificación	C2 — Configura servicios de correo electrónico, TELNET y SSH (40 %).
Docente	M.Sc. Ing. Jimmy Nataniel Requena Llorentty
Estudiante	Lizeth Rodríguez · Fecha: 25 / 10 / 2026
Resultado	`verificar_lab10.sh lizeth` → **11 de 11 comprobaciones** ✅

1. Objetivo

Demostrar con capturas reales la diferencia entre TELNET y SSH, regenerar y verificar la identidad del servidor, configurar el acceso por clave ed25519 con `known_hosts` y `~/.ssh/config`, y publicar un archivo por `scp`, hasta pasar el verificador de 11 comprobaciones. Este informe documenta cómo se hizo, qué se observó en cada prueba y por qué.

2. Contexto del problema

El técnico anterior administraba el servidor del colegio con TELNET "porque era más simple". La directora quiere entender el riesgo con una prueba concreta y que, desde ahora, el equipo de TI entre solo con claves, sin contraseñas viajando por la red.

Escenario de la práctica

```
TELNET :23 ➜ usuario, contraseña y comandos en texto plano
SSH    :22 ➜ kex sntrup761x25519 + chacha20-poly1305 (todo cifrado)

root (cliente) ── ssh horizonte ──➜ lizeth@srv-horizonte   (clave id_ed25519)
               ── scp/sftp/rsync ➜ /srv/horizonte/www     (root:webadmin 2775)
```

3. Requisitos previos

Se usó el servidor del colegio de los labs 2 al 5: el contenedor `horizonte-servidor-lab02` (hostname `srv-horizonte`), con los puertos 2222→22, 8080→80 y 8443→443 publicados en Windows (Figura 1), y la administradora del Lab 2, `lizeth`, miembro del grupo `webadmin`. La carpeta `lab10` se copió al servidor con

`docker cp` y los scripts se normalizaron con `sed -i 's/\r$//'`. Para abrir los `.pcap` se instaló **Wireshark 4.4** en Windows.

Preparación del entorno

```
docker cp $HOME\Documents\Acti-3-tec-wed-1\lab10 horizonte-servidor-lab02:/root/lab10
docker exec -it horizonte-servidor-lab02 bash
cd ~/lab10 && sed -i 's/\r$//' *.sh
ssh -V        → OpenSSH_9.6p1 Ubuntu-3ubuntu13.19, OpenSSL 3.0.13
```

Figura 1

El servidor del colegio activo, con SSH publicado en el puerto 2222



4. Archivos del laboratorio

Tabla 1

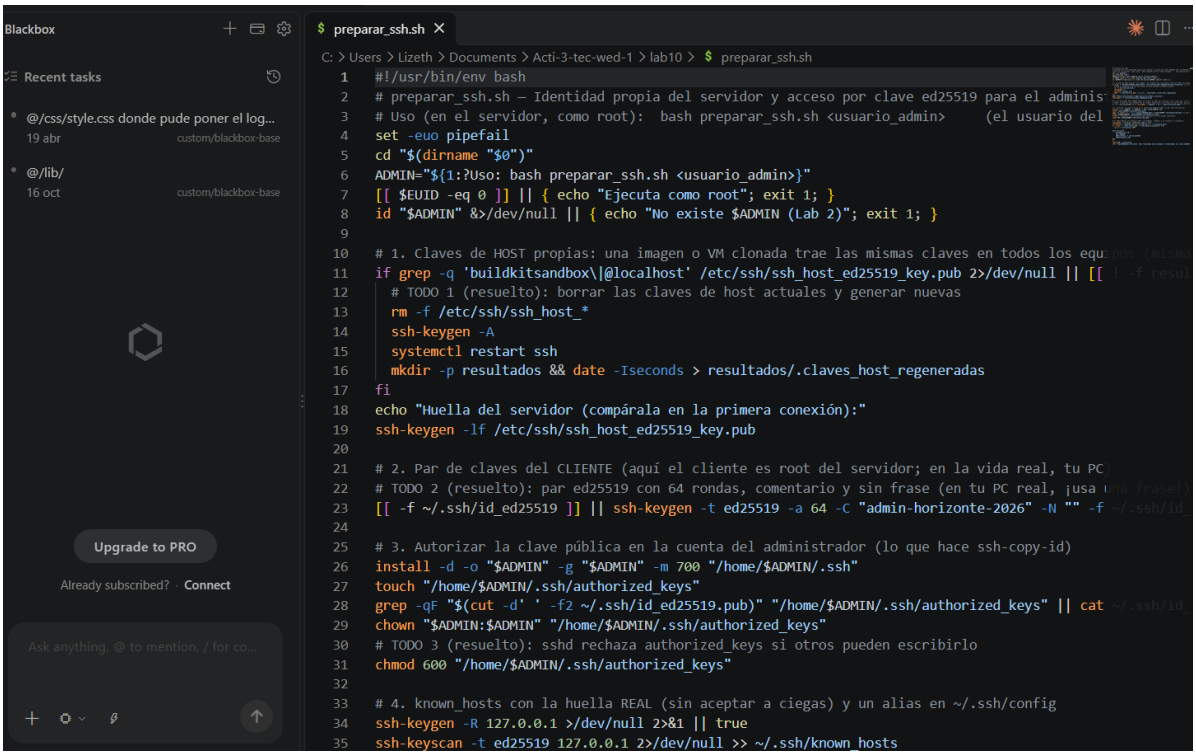
Archivos del Laboratorio y TODO Completados

Archivo	TODO	Qué se completó
`preparar_ssh.sh`	4	TODO 1: `rm -f /etc/ssh/ssh_host_*` y `ssh-keygen -A` · TODO 2: par ed25519 `-a 64 -C "admin-horizonte-2026"` · TODO 3: `chmod 600` a `authorized_keys` · TODO 4: `IdentityFile` e `IdentitiesOnly yes` en el alias.
`capturar.sh`	—	Sesión automática con `expect` mientras `tcpdump` graba el puerto 23 o 22 (completo).
`telnet_laboratorio.sh`	—	Activa TELNET solo para la demostración y lo desinstala al final (completo).
`comunicado.html`	—	Publicado por `scp`; permisos corregidos a 664.
`verificar_lab10.sh`	—	11 comprobaciones; resultado final **11 de 11**.

Al pegar los TODO en VS Code se borraron por error las líneas `EOF` y `fi` que cierran el bloque `Host horizonte`; sin ellas el script daba error de sintaxis. Se reemplazó el archivo completo antes de ejecutarlo (Figura 2).

Figura 2

preparar_ssh.sh con los TODO 1 a 3 resueltos (el TODO 4 sigue más abajo)



5. Desarrollo paso a paso

Paso 1 — La prueba del espía

Parte A — capturas reales

Antes de empezar, el servidor tenía la clave de host `root@buildkitsandbox` (Figura 3), dato que se usa en el Paso 2. TELNET **no estaba instalado**: `/etc/inetd.conf` no existía y solo escuchaba SSH. `telnet_laboratorio.sh activar` instaló `telnetd` con `openbsd-inetd`. Ubuntu deja la línea desactivada (`#<off>#`) y el script la habilitó con `update-inetd --enable telnet`. Así quedaron `inetd` en el puerto **23** y `sshd` en el **22** (Figura 4).

Figura 3

OpenSSH 9.6p1 y la clave de host original: root@buildkitsandbox

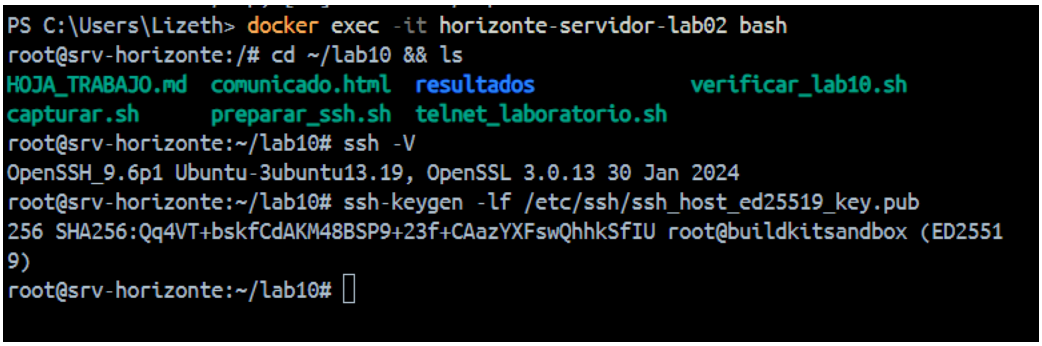
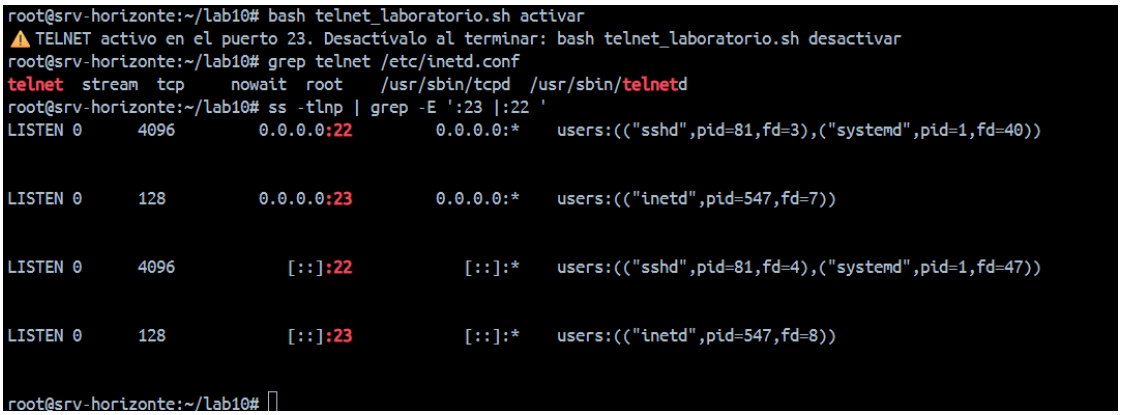


Figura 4

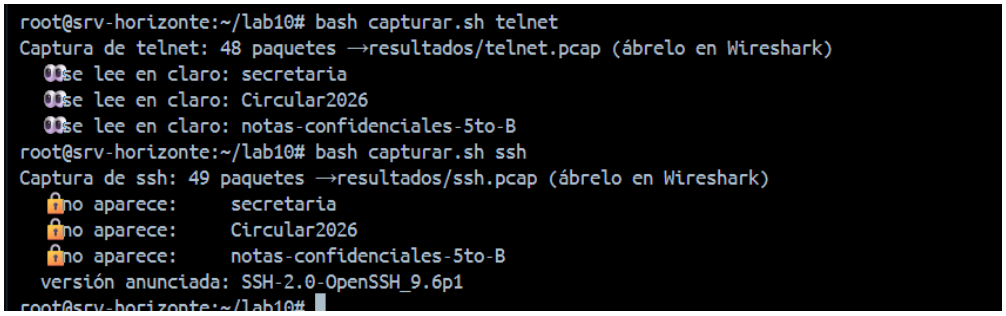
TELNET activado: línea en inetd.conf y puertos 22 y 23 abiertos



`capturar.sh` inicia una sesión con la usuaria de prueba `secretaria`, escribe un texto "confidencial" y la cierra, mientras `tcpdump` graba el tráfico. Luego busca esos datos dentro de la captura (Figura 5).

Figura 5

La prueba del espía: TELNET deja todo legible; SSH, nada



Nota. La primera ejecución dio 47 paquetes TELNET y la repetición 48; el resultado fue el mismo en las dos.

Tabla 2

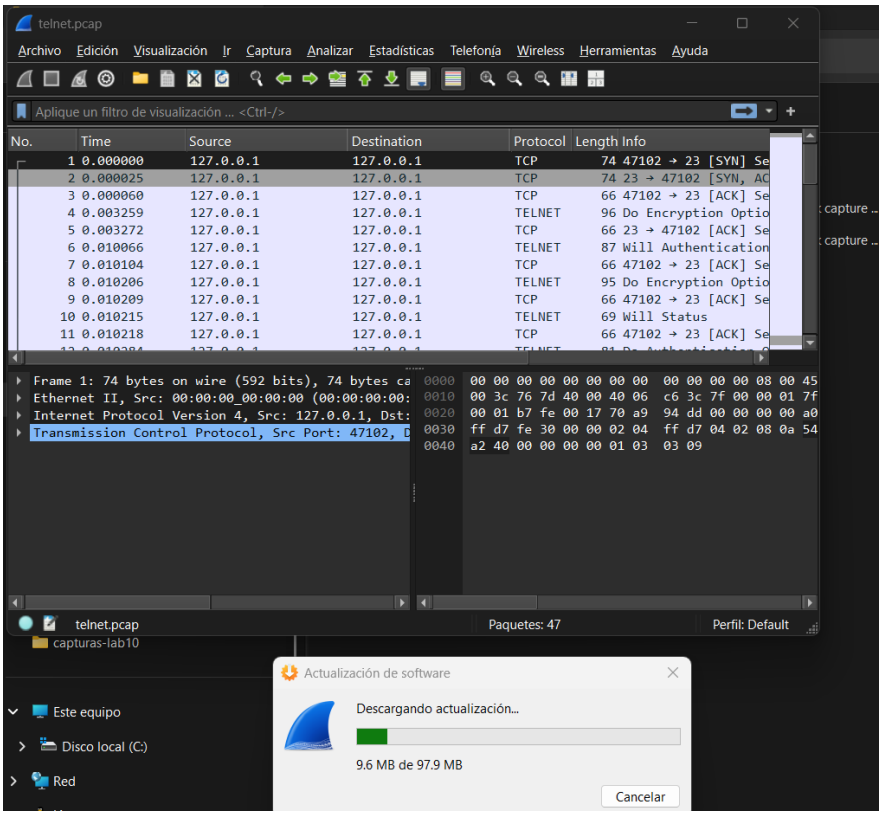
Parte A: Qué Ve un Espía en Cada Protocolo

	TELNET (puerto 23)	SSH (puerto 22)
¿Se lee el usuario?	**Sí**: `secretaria`	No
¿Se lee la contraseña?	**Sí**: `Circular2026`	No
¿Se lee lo que se escribió en la sesión?	**Sí**: `notas-confidenciales-5to-B`	No
¿Qué se ve en claro?	Todo: banner del sistema, `login:`, `Password:`, usuario, contraseña, comandos y respuestas	Solo la versión `SSH-2.0-OpenSSH_9.6p1` y la lista de algoritmos que se negocian

Para verlo como lo vería un atacante, los `.pcap` se copiaron a Windows y se abrieron en Wireshark (Figura 6). Con **Analizar → Seguir → Flujo TCP**, la sesión TELNET se lee completa (Figura 7): en azul, el servidor envía el banner `Linux … (srv-horizonte)`, `srv-horizonte login:` y `Password:`; en rojo, el cliente responde `secretaria` y `Circular2026`. En SSH (Figura 8), lo único legible es el intercambio de versiones y la lista de algoritmos; todo lo demás son bytes cifrados.

Figura 6

telnet.pcap abierto en Wireshark: 47 paquetes en el puerto 23



Nota. Wireshark tenía abierto `telnet.pcap` cuando se intentó copiar la segunda captura (`unlinkat … being used by another process`), así que se analizó la de la primera ejecución.

Figura 7

Seguir flujo TCP en TELNET: usuario y contraseña en texto plano

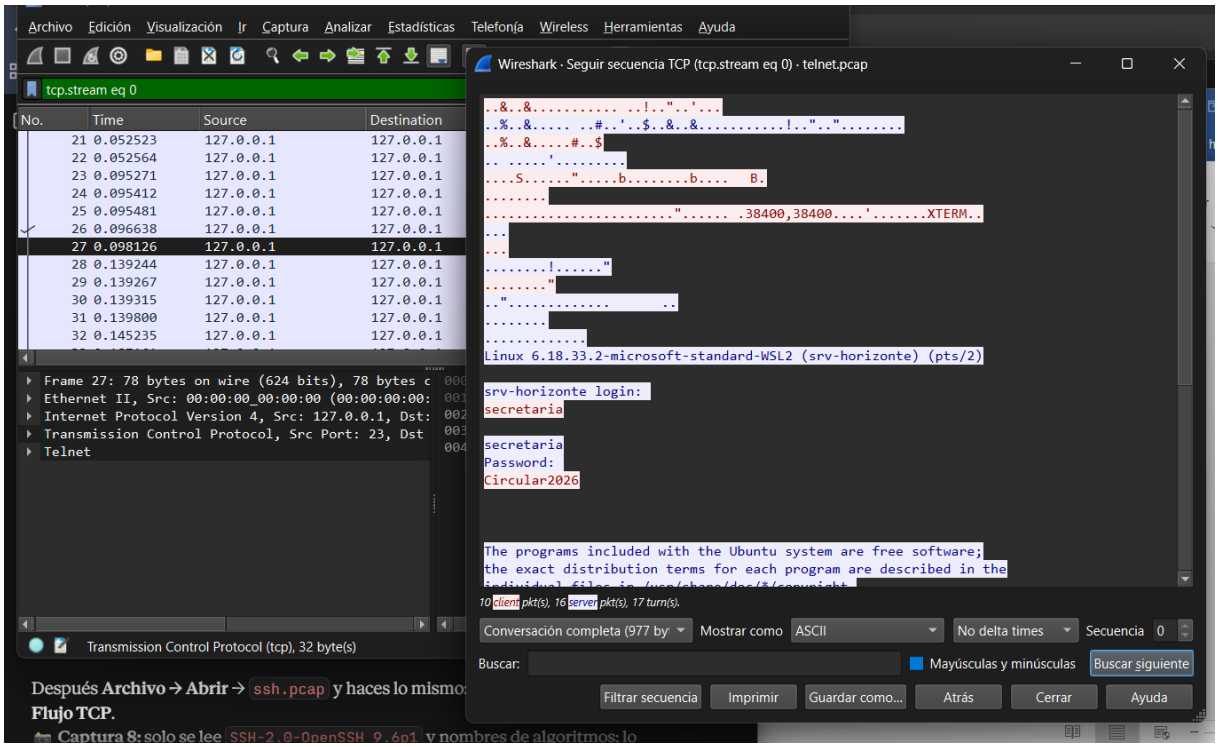
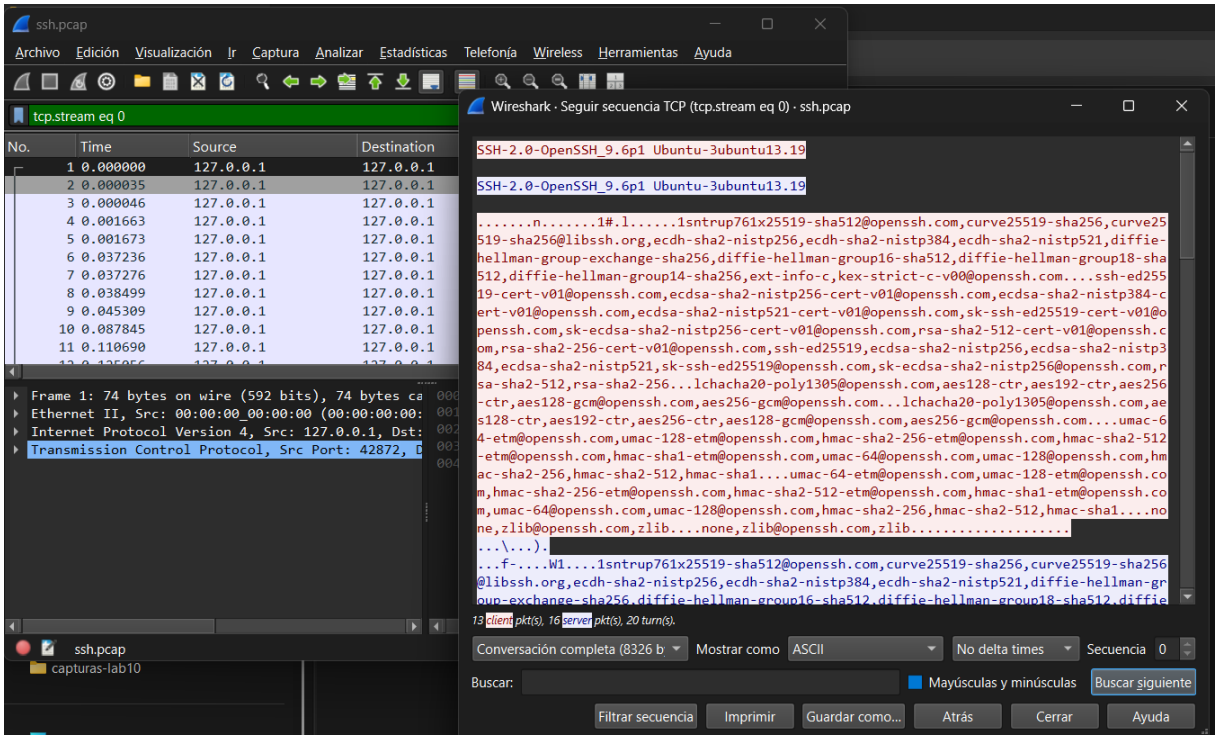


Figura 8

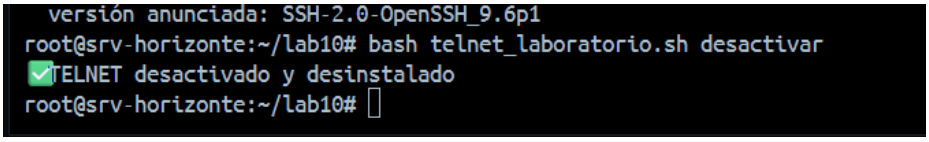
Seguir flujo TCP en SSH: solo versión y algoritmos; el resto, cifrado



Terminada la demostración, TELNET se desactivó y se **desinstaló** (Figura 9). Todavía existe en algunos routers, impresoras y equipos industriales por herencia: es muy liviano, muchos equipos se fabricaron antes de SSH y nunca recibieron otro firmware. Donde no se puede reemplazar, hay que aislarlo en una red de gestión.

Figura 9

TELNET desactivado y desinstalado



> 🔎 **Verificado**
>
> En TELNET se lee `Circular2026`; en SSH, nada salvo la versión.

Paso 2 — La identidad del servidor

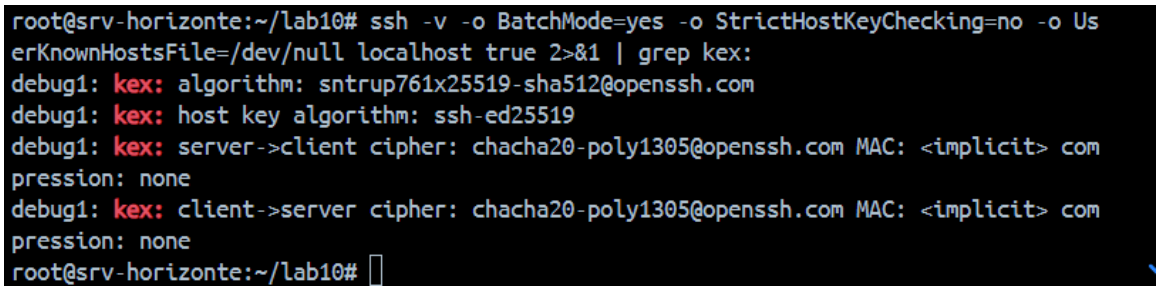
Parte B — clave de host y kex

La huella inicial terminaba en `root@buildkitsandbox`: la clave se creó al **construir la imagen**, así que todos los que usan esa imagen comparten la misma "cédula" de servidor. Si 30 estudiantes o 30 VMs clonadas tienen la misma clave de host, cualquiera tiene la privada y puede hacerse pasar por el servidor de otro (*man-in-the-middle*) sin que el cliente note nada, porque la huella es la esperada.

`ssh -v` mostró los algoritmos negociados (Figura 10). El intercambio de claves es **`sntrup761x25519-sha512`**, un esquema **híbrido post-cuántico**: combina *Streamlined NTRU Prime 761* (retículos, resistente a computadoras cuánticas) con **X25519** (curva elíptica clásica). Para descifrar habría que romper los dos, y así se protege del ataque "guardar ahora, descifrar después". El servidor se identifica con **ssh-ed25519** y la sesión se cifra con **ChaCha20-Poly1305**, un cifrado autenticado (por eso el MAC aparece como `<implicit>`).

Figura 10

Algoritmos negociados: kex híbrido post-cuántico y ChaCha20-Poly1305



Nota. Esta consulta de diagnóstico a `localhost` se hizo antes de preparar `known_hosts`, por eso lleva `StrictHostKeyChecking=no`; esa opción no quedó en ninguna configuración.

El TODO 1 borra las claves de host y genera unas nuevas con `ssh-keygen -A`. Al ejecutar `preparar_ssh.sh lizeth` aparecieron `generating new host keys: RSA ECDSA ED25519` y la nueva huella **`SHA256:KueRdm7E… root@srv-horizonte`**, distinta de la de la imagen (Figura 11).

`preparar_ssh.sh — TODO 1`

```
rm -f /etc/ssh/ssh_host_*
ssh-keygen -A
systemctl restart ssh
```

> 🔎 **Verificado**
>
> `ssh-keygen -lf` muestra `root@srv-horizonte` (ya no `buildkitsandbox`).

Paso 3 — Acceso con clave ed25519

Parte C — preparar_ssh.sh

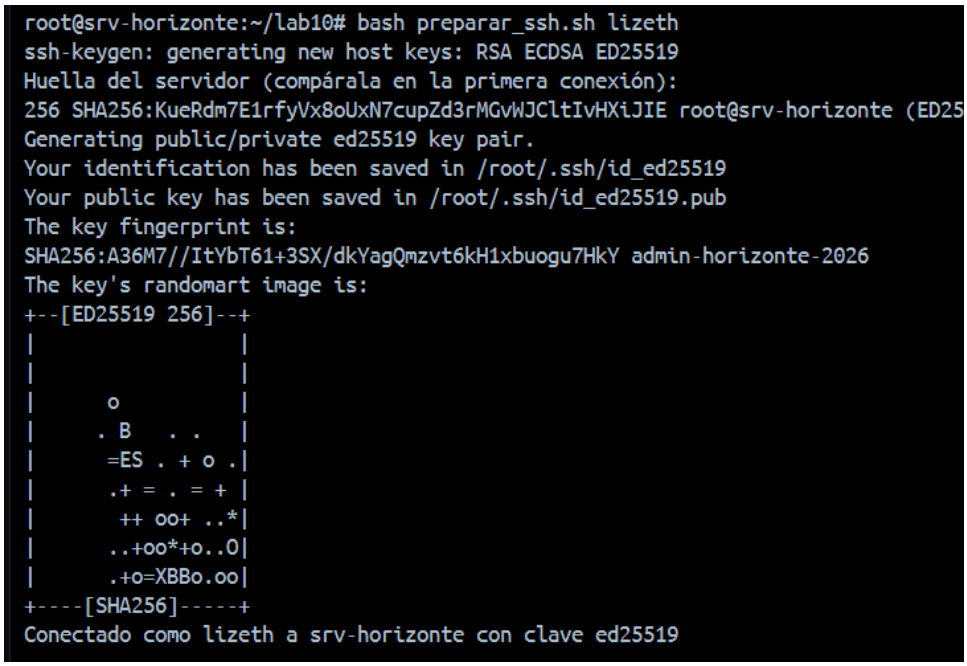
`preparar_ssh.sh — TODO 2 a 4`

```
[[ -f ~/.ssh/id_ed25519 ]] || ssh-keygen -t ed25519 -a 64 -C "admin-horizonte-2026" -N "" -f
~/.ssh/id_ed25519   # TODO 2
chmod 600 "/home/$ADMIN/.ssh/authorized_keys"
# TODO 3
Host horizonte
    HostName 127.0.0.1
    User lizeth
    IdentityFile ~/.ssh/id_ed25519
# TODO 4
    IdentitiesOnly yes
```

El script generó el par `admin-horizonte-2026`, copió la clave pública en `authorized_keys` de lizeth, agregó a `known_hosts` la huella **real** con `ssh-keyscan` (sin aceptar a ciegas) y terminó con **`Conectado como lizeth a srv-horizonte con clave ed25519`** (Figura 11).

Figura 11

Claves de host nuevas, par ed25519 del cliente y conexión sin contraseña

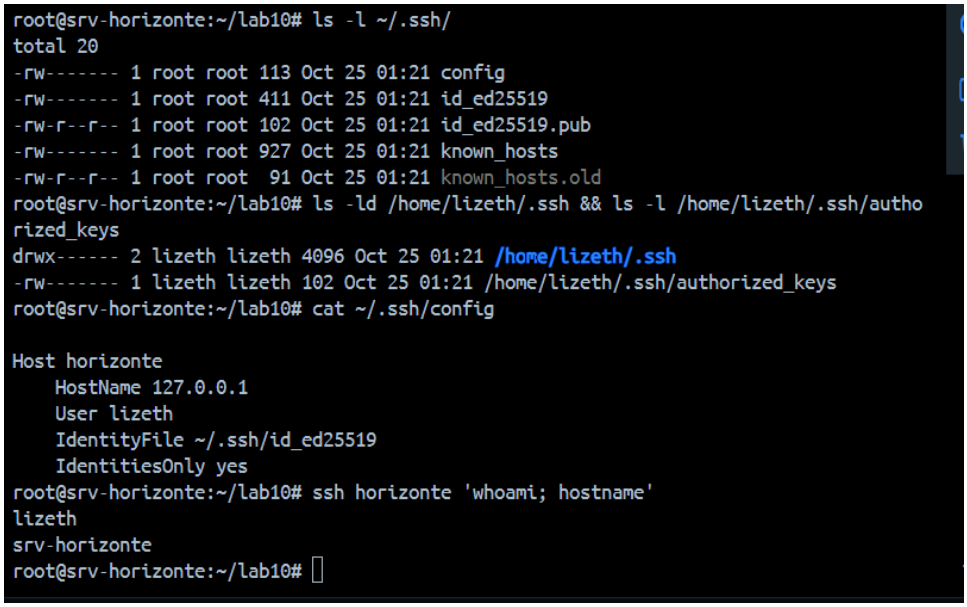


En la Figura 12 se comprueban los permisos. `id_ed25519` (la **privada**) tiene 600 y **nunca sale del equipo cliente**; solo se listó, nunca se abrió. `id_ed25519.pub` (la **pública**) tiene 644 y es la que se copia a los

servidores. En la cuenta de lizeth, `.ssh` quedó con **700** y `authorized_keys` con **600**: si otros pudieran escribir ese archivo, podrían agregar su clave y entrar como ella, por eso `sshd` (`StrictModes`) lo ignora si los permisos son abiertos. `ssh horizonte` entró como `lizeth` **sin pedir contraseña**: `IdentitiesOnly yes` hace que use solo esa clave.

Figura 12

Permisos 700/600, alias horizonte y conexión con clave



> 🔎 **Verificado**
>
> "Conectado como lizeth … con clave ed25519" sin pedir contraseña.

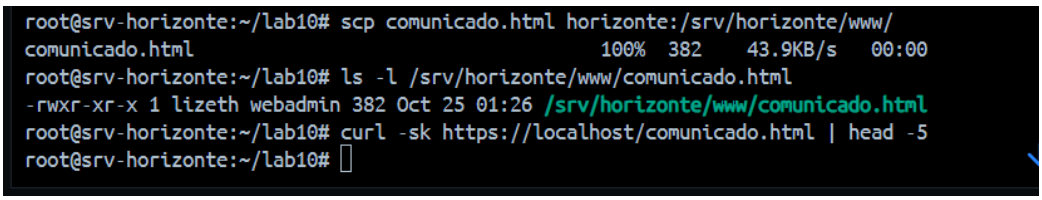
Paso 4 — Publicar con scp

Parte D — transferencia segura

`scp comunicado.html horizonte:/srv/horizonte/www/` publicó el archivo con dueño **`lizeth`** (`scp` entra como la usuaria del alias) y grupo **`webadmin`**. El grupo se hereda porque en el Lab 2 la carpeta quedó `root:webadmin 2775`: el bit **setgid** (la `s` de `drwxrwsr-x`) hace que todo lo creado dentro tome el grupo de la carpeta. Pero quedó `-rwxr-xr-x`, porque vino de Windows con 755 vía `docker cp` y `scp` conserva los permisos, y el primer `curl` a `localhost` no devolvió nada (Figura 13).

Figura 13

Primera publicación: dueño y grupo correctos, pero permisos 755



Se corrigió con `chmod 664` y `scp -p`, y quedó `-rw-rw-r-- lizeth webadmin`, igual que el resto del sitio (Figura 14). El `curl` a `localhost` daba `000` porque desde el Lab 5 nginx solo responde HTTPS al nombre del colegio. Pidiendo `www.horizonte.internal` devolvió el HTML.

Figura 14

Permisos corregidos y nginx sirviendo el comunicado por su nombre

```
root@srv-horizonte:~/lab10# chmod 664 comunicado.html
root@srv-horizonte:~/lab10# scp -p comunicado.html horizonte:/srv/horizonte/www/ && l
s -l /srv/horizonte/www/comunicado.html
comunicado.html                               100%  382   180.5KB/s   00:00
-rw-rw-r-- 1 lizeth webadmin 382 Sep 30 10:38 /srv/horizonte/www/comunicado.html
root@srv-horizonte:~/lab10# systemctl is-active nginx
active
root@srv-horizonte:~/lab10# curl -sk -o /dev/null -w "%{http_code}\n" https://localho
st/comunicado.html
000
root@srv-horizonte:~/lab10# curl -sk --resolve www.horizonte.internal:443:127.0.0.1 h
ttps://www.horizonte.internal/comunicado.html | head -5
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
root@srv-horizonte:~/lab10#
```

`sftp` (en modo lote) listó el sitio y `rsync -av` transfirió el archivo, ambos por el mismo canal SSH y con la misma clave (Figuras 15 y 16). Finalmente, el comunicado se abrió en el navegador de Windows por el puerto publicado 8443 (Figura 17).

Figura 15

sftp horizonte: listado de /srv/horizonte/www

```
root@srv-horizonte:~/lab10# echo "ls -l /srv/horizonte/www" | sftp -b - horizonte
sftp> ls -l /srv/horizonte/www
-rw-rw-r--    ? root     webadmin      464 Oct  9 22:25 /srv/horizonte/www/404.html
-rw-rw-r--    ? root     webadmin      505 Oct  9 22:25 /srv/horizonte/www/50x.html
drwxrwsr-x    ? root     webadmin     4096 Oct  9 22:17 /srv/horizonte/www/circulares
-rwxr-xr-x    ? lizeth   webadmin      382 Oct 25 01:26 /srv/horizonte/www/comunicado
.html
drwxrwsr-x    ? root     webadmin     4096 Oct  9 22:17 /srv/horizonte/www/css
-rw-r--r--    ? root     webadmin      830 Oct 12 12:39 /srv/horizonte/www/horizonte-
ca.crt
-rw-rw-r--    ? root     webadmin      917 Oct  9 22:25 /srv/horizonte/www/index.html
-rw-rw-r--    ? lizeth   webadmin        0 Oct  7 12:51 /srv/horizonte/www/prueba.txt
root@srv-horizonte:~/lab10# command -v rsync >/dev/null || apt-get install -y -qq rsy
nc
debconf: delaying package configuration, since apt-utils is not installed
Selecting previously unselected package libpopt0:amd64.
(Reading database ... 17565 files and directories currently installed.)
Preparing to unpack .../libpopt0_1.19+dfsg-1build1_amd64.deb ...
Unpacking libpopt0:amd64 (1.19+dfsg-1build1) ...
Selecting previously unselected package rsync.
Preparing to unpack .../rsync_3.2.7-1ubuntu1.5_amd64.deb ...
Unpacking rsync (3.2.7-1ubuntu1.5) ...
```

Nota. El listado se tomó antes de corregir los permisos; el archivo final quedó 664 (Figura 14).

Figura 16

rsync -av sobre SSH

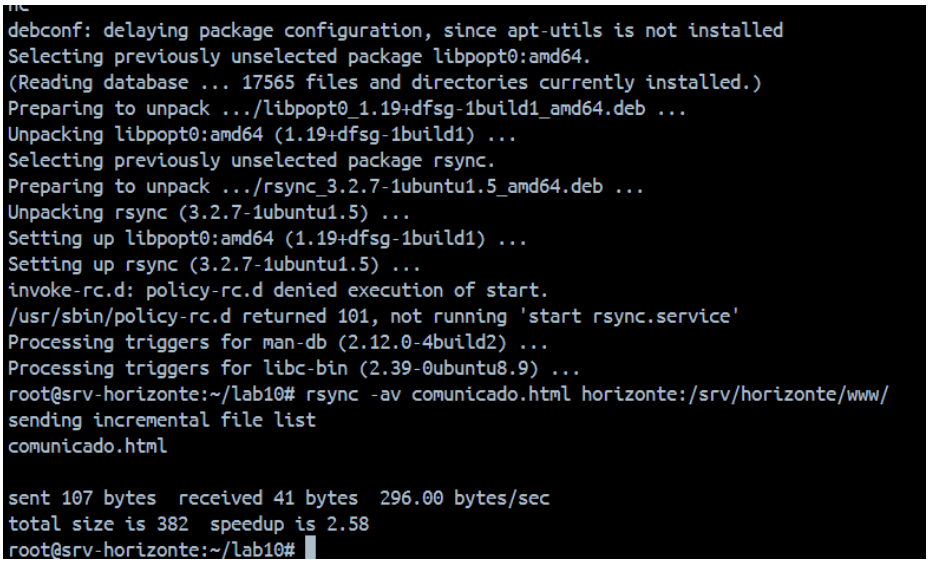
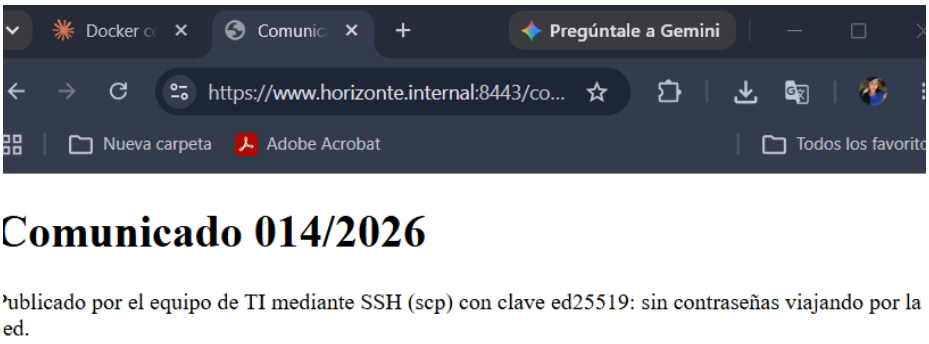


Figura 17

Comunicado 014/2026 publicado y visible en https://www.horizonte.internal:8443



> 🔎 **Verificado**
>
> `comunicado.html` queda con dueño `lizeth` y grupo `webadmin`.

Paso 5 — Cierre seguro

Con TELNET ya desinstalado, el verificador aprobó las **11 comprobaciones** (Figura 18). Por último, la carpeta `resultados/` (las dos capturas `.pcap`, sus resúmenes `.txt` y la marca de claves regeneradas) se copió a la carpeta `lab10` del proyecto para la entrega (Figura 19).

Figura 18

verificar_lab10.sh: 11 de 11 comprobaciones

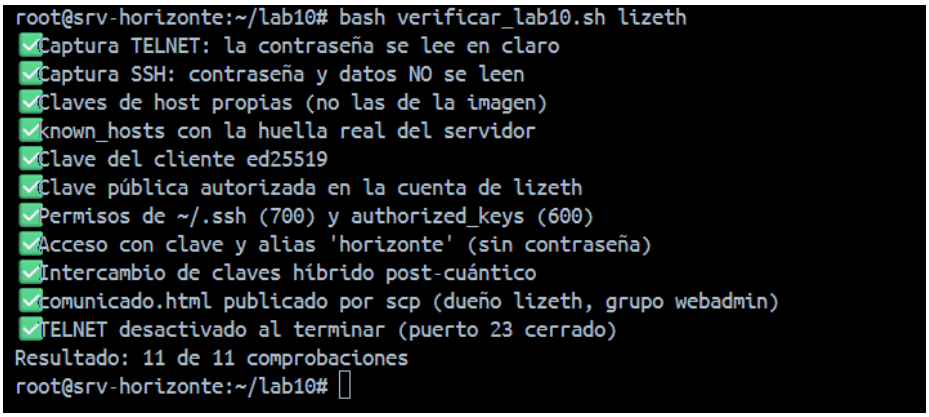
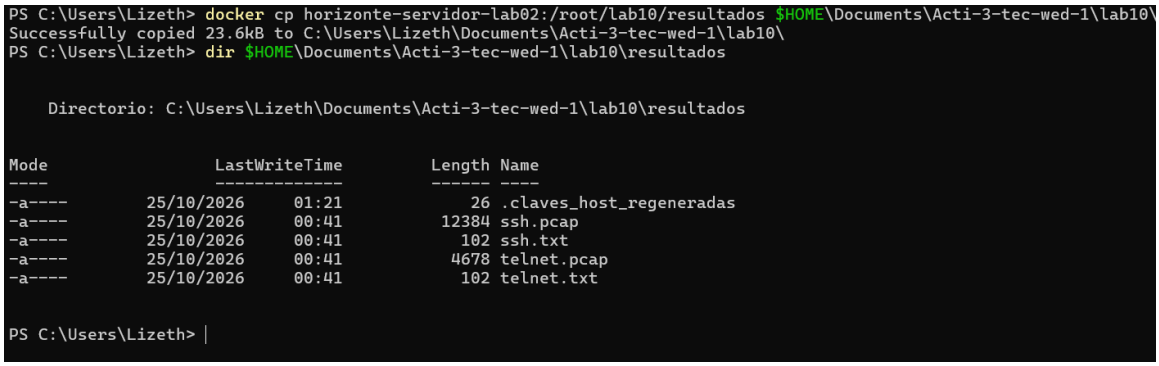


Figura 19

Carpeta resultados copiada para el entregable



Reflexión — "REMOTE HOST IDENTIFICATION HAS CHANGED"

Ese aviso significa que la clave del servidor no coincide con la guardada en `known_hosts`. Puede ser un cambio legítimo (como la regeneración del Paso 2) o alguien interceptando la conexión. **Qué hago:** detenerme sin escribir nada, verificar la huella por otro canal (consola del servidor o el administrador) con `ssh-keygen -lf`, compararla con la del aviso y, solo si coincide, borrar la entrada vieja con `ssh-keygen -R <host>`. **Qué NO hago:** borrar todo `known_hosts`, usar `StrictHostKeyChecking=no` o aceptar la huella nueva sin verificarla, porque eso apaga justo la protección contra el ataque de intermediario.

> 🔎 **Verificado**
>
> TELNET desactivado · **Resultado: 11 de 11 comprobaciones.**

6. Retos (para llegar a Estratégico)

Los retos opcionales **no se realizaron** en esta entrega. Se dejan indicados con lo que haría falta para cada uno.

Tabla 3

Estado de los Retos

Reto	Estado	Qué haría falta
Registro SSHFP en `horizonte.internal` y `VerifyHostKeyDNS`	No realizado	Generar los registros con `ssh-keygen -r srv-horizonte`, agregarlos a la zona del Lab 7 (subiendo el serial) y conectarse con `-o VerifyHostKeyDNS=yes`.
`ssh-agent` con clave protegida por frase	No realizado	Poner frase con `ssh-keygen -p`, cargarla una vez con `ssh-add` y contar cuántas veces la pide en 5 conexiones (una sola).
CA de SSH que firma la clave de host	No realizado	Crear la CA con `ssh-keygen`, firmar la clave de host con `ssh-keygen -s ca -h`, usar `HostCertificate` en sshd y `@cert-authority` en `known_hosts`.

7. Uso de Inteligencia Artificial en este laboratorio

✅ Permitido — cómo se usó

- Explicación del saludo SSH y de cada algoritmo negociado (`sntrup761x25519`, `ssh-ed25519`, `chacha20-poly1305`).
- Diagnóstico de problemas concretos: el `EOF` borrado, Wireshark ausente, permisos 755 y el `000` de `curl`.
- Comparación de ed25519, ECDSA y RSA para la clave del cliente.

⛔ No permitido — cómo se respetó

- La clave privada **no** se pegó en la IA ni aparece en ninguna captura: solo su nombre y permisos con `ls -l`.
- No se aceptaron huellas a ciegas: `known_hosts` se llenó con `ssh-keyscan` después de regenerar y comparar la huella.
- TELNET se **desinstaló** al terminar (verificador: puerto 23 cerrado).

La bitácora completa, con las 9 consultas realizadas, cómo se verificó cada una y qué se aprendió, está en el **Anexo B**.

8. Entregable

Tabla 4

Lista de Comprobación del Entregable

Elemento	Estado	Evidencia
`preparar_ssh.sh` completo	✅	4 de 4 TODO (Tabla 1, Figura 2)
Carpeta `resultados/` (`.pcap` y `.txt`)	✅	Figura 19
Captura del verificador con 11 de 11	✅	Figura 18
Captura de Wireshark con la sesión TELNET	✅	Figura 7 (y SSH en la Figura 8)
`HOJA_TRABAJO.md` (partes A a E)	✅	Anexo A

Elemento	Estado	Evidencia
`BITACORA_IA.md`	✅	Anexo B

9. Rúbrica de evaluación: evidencias por criterio

Tabla 5

Criterios de la Rúbrica y Dónde se Evidencian

Criterio	Puntos	Evidencia en este informe
Prueba del espía	25	Capturas TELNET y SSH (Figuras 5 a 8) con la tabla completa y explicada (Tabla 2).
Identidad del servidor	20	Problema de `buildkitsandbox` explicado; claves regeneradas (Figura 11); kex analizado (Figura 10); aviso de MITM (Reflexión).
Acceso con clave	25	Par ed25519, `authorized_keys` 600 y `.ssh` 700, alias en `~/.ssh/config` (Figuras 11 y 12).
Transferencia segura	15	`scp` con dueño `lizeth` y grupo `webadmin` explicado por el setgid; `sftp`, `rsync` y navegador (Figuras 13 a 17).
Cierre y bitácora	15	TELNET desinstalado (Figura 9) y verificador 11/11 (Figura 18); bitácora verificada (Anexo B).

10. Conclusiones

La prueba del espía lo deja sin discusión: con TELNET, cualquiera que capture la red lee el usuario, la contraseña y todo lo que se escribe, como se ve en Wireshark. Con SSH, la misma sesión solo muestra la versión del programa y la lista de algoritmos. El intercambio de claves negociado es incluso post-cuántico. TELNET quedó desinstalado y el servidor ahora tiene su propia identidad, en lugar de la clave compartida de la imagen de Docker.

El equipo de TI ya entra con clave ed25519 y un alias, sin que ninguna contraseña viaje por la red, y publica en el sitio con `scp`, `sftp` o `rsync` respetando el grupo `webadmin` del Lab 2. Queda pendiente, para la siguiente práctica, cerrar del todo la puerta de las contraseñas: desactivar `PasswordAuthentication` y el acceso de root en `sshd_config`. Mientras tanto, la clave privada debe protegerse con una frase en la PC real de cada administrador.

Referencias

Bernstein, D. J., Chuengsatiansup, C., Lange, T., & van Vredendaal, C. (2017). *NTRU Prime: Reducing attack surface at low cost*. En C. Adams & J. Camenisch (Eds.), *Selected Areas in Cryptography – SAC 2017* (pp. 235–260). Springer. https://doi.org/10.1007/978-3-319-72565-9_12

OpenBSD Project. (2022). *OpenSSH 9.0 release notes*. OpenSSH. https://www.openssh.com/txt/release-9.0

Postel, J., & Reynolds, J. (1983). *Telnet protocol specification* (RFC 854). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc854

Requena Llorentty, J. N. (2026). *Guía de laboratorio 10: TELNET vs SSH: la prueba del espía, la identidad del servidor y el acceso con claves ed25519* [Guía de laboratorio]. Universidad Privada Domingo Savio.

Wireshark Foundation. (s.f.). *Following protocol streams*. Wireshark User's Guide. https://www.wireshark.org/docs/wsug_html_chunked/ChAdvFollowStreamSection.html

Ylonen, T., & Lonvick, C. (Ed.). (2006). *The Secure Shell (SSH) transport layer protocol* (RFC 4253). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc4253

Anexo A — HOJA_TRABAJO.md

Laboratorio 10 — Hoja de trabajo: TELNET vs SSH, "la prueba del espía".

Nombre: Lizeth Rodríguez Fecha: 25/10/2026

Servidor del colegio: contenedor `horizonte-servidor-lab02` (hostname `srv-horizonte`, Ubuntu 24.04, OpenSSH_9.6p1 Ubuntu-3ubuntu13.19), como root. Administradora del Lab 2: `lizeth` (grupo `webadmin`). Usuaria de prueba que crean los scripts: `secretaria` / `Circular2026` (solo para el laboratorio).

Parte A — La prueba del espía

1. ¿En qué estado venía TELNET en Ubuntu 24.04? No estaba instalado: antes de activar, `/etc/inetd.conf` ni siquiera existía y solo escuchaba SSH (puerto 22). Al instalar `telnetd` + `openbsd-inetd`, Ubuntu agrega la línea de telnet **desactivada** (`#<off># telnet …`). El script la habilitó con `update-inetd --enable telnet`, y después quedó así:

```
telnet  stream  tcp     nowait  root    /usr/sbin/tcpd  /usr/sbin/telnetd
```

`ss -tlnp` mostró `inetd` escuchando en el **23** y `sshd` en el **22**.

2. Capturas (`bash capturar.sh telnet` → 47 paquetes, y 48 al repetirlo · `bash capturar.sh ssh` → 49 paquetes, las dos veces):

	TELNET (puerto 23)	SSH (puerto 22)
¿Se lee el usuario?	**Sí**: `secretaria`	No
¿Se lee la contraseña?	**Sí**: `Circular2026`	No
¿Se lee lo que se escribió en la sesión?	**Sí**: `notas-confidenciales-5to-B`	No
¿Qué se ve en claro?	Todo: banner del sistema, `login:`, `Password:`, usuario, contraseña, comandos y respuestas	Solo la versión (`SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.19`) y la lista de algoritmos que se negocian; el resto va cifrado

3. Wireshark. Se copiaron `telnet.pcap` y `ssh.pcap` a Windows con `docker cp` y se abrieron con *Analizar > Seguir > Flujo TCP*. En TELNET, el servidor (azul) envía `srv-horizonte login:` y `Password:`, y el cliente (rojo) responde `secretaria` y `Circular2026` letra por letra, legibles. En SSH, después del intercambio de versiones y de la lista de algoritmos (`sntrup761x25519-sha512@openssh.com`, `curve25519-sha256`, `chacha20-poly1305@openssh.com`, …), solo hay bytes cifrados. Buscar `Circular2026` en el flujo SSH no da resultados.

4. `bash telnet_laboratorio.sh desactivar` → ✅ `TELNET desactivado y desinstalado` (puerto 23 cerrado).

¿Por qué existe todavía TELNET en algunos equipos (routers, impresoras)? Por herencia y por costo. Es un protocolo muy simple que cabe en equipos con poca memoria y poco procesador, muchos equipos viejos se fabricaron cuando no existía SSH y el fabricante nunca actualizó el firmware, y algunos administradores lo mantienen "porque siempre funcionó". También se usa `telnet IP puerto` como herramienta para probar si un puerto responde. Lo correcto es desactivarlo, usar SSH donde exista o, si no hay alternativa, aislar ese equipo en una red de gestión.

Parte B — ¿Con quién estoy hablando? (claves de host)

1. `ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub` → `256 SHA256:Qq4VT+bskfCdAKM48BSP9+23f+CAazYXFswQhhkSfIU root@buildkitsandbox (ED25519)`

El comentario `root@buildkitsandbox` indica que la clave se generó **al construir la imagen de Docker**, no en mi servidor. Si 30 estudiantes (o 30 VMs clonadas) tienen la **misma** clave de host, todos comparten la misma "cédula": cualquiera de ellos tiene la clave privada y puede hacerse pasar por el servidor de otro (ataque de intermediario, *man-in-the-middle*), y el cliente no lo notaría porque la huella es la esperada. Además, la huella ya no identifica a un equipo concreto.

2. TODO 1 (regenerar las claves de host):

```
rm -f /etc/ssh/ssh_host_*
ssh-keygen -A
```

Resultado: `ssh-keygen: generating new host keys: RSA ECDSA ED25519` y la nueva huella `256 SHA256:KueRdm7E1rfyVx8oUxN7cupZd3rMGvWJCltIvHXiJIE root@srv-horizonte (ED25519)`.

3. `ssh -v … localhost true 2>&1 | grep kex:`

```
kex: algorithm: sntrup761x25519-sha512@openssh.com
kex: host key algorithm: ssh-ed25519
kex: server->client cipher: chacha20-poly1305@openssh.com MAC: <implicit> compression: none
kex: client->server cipher: chacha20-poly1305@openssh.com MAC: <implicit> compression: none
```

sntrup761x25519 es un intercambio de claves **híbrido post-cuántico**: combina *Streamlined NTRU Prime 761* (un algoritmo de retículos pensado para resistir computadoras cuánticas) con **X25519** (curva elíptica clásica). La clave de sesión depende de los dos, así que hay que romper ambos para descifrar. Protege contra "guardar ahora, descifrar después": aunque alguien grabe hoy el tráfico, una futura computadora cuántica no le bastaría. Es el predeterminado desde OpenSSH 9.0. El servidor se identificó con su clave **ed25519** y la sesión se cifra con **ChaCha20-Poly1305** (cifrado autenticado, por eso el MAC es `<implicit>`).

Parte C — Acceso con clave ed25519

TODO 2 a 4 de `preparar_ssh.sh`:

```
[[ -f ~/.ssh/id_ed25519 ]] || ssh-keygen -t ed25519 -a 64 -C "admin-horizonte-2026" -N "" -f
~/.ssh/id_ed25519
chmod 600 "/home/$ADMIN/.ssh/authorized_keys"
    IdentityFile ~/.ssh/id_ed25519
    IdentitiesOnly yes
```

`bash preparar_ssh.sh lizeth` → claves de host nuevas, par `admin-horizonte-2026` (`SHA256:A36M7//ItYbT61+3SX/dkYagQmzvt6kH1xbuogu7HkY`) y **`Conectado como lizeth a srv-horizonte con clave ed25519`**.

Al copiar el script se habían borrado sin querer las líneas `EOF` y `fi` que cierran el bloque `Host horizonte`. Se corrigió antes de ejecutarlo.

1. Diferencia entre `id_ed25519` e `id_ed25519.pub`. `id_ed25519` es la clave **privada** (permisos 600, solo root) y **nunca sale de la PC** del administrador: no se copia, no se envía y no se pega en ningún lado. `id_ed25519.pub` es la clave **pública** (644) y es la que se copia a los servidores. Con la pública el servidor comprueba una firma que solo la privada puede hacer, así que la contraseña ya no viaja.

2. ¿Qué hace `authorized_keys`? ¿Por qué 600? Es la lista de claves públicas que pueden entrar a esa cuenta (`/home/lizeth/.ssh/authorized_keys`). Si otros usuarios pudieran escribirlo, podrían agregar su propia clave y entrar como lizeth. Por eso `sshd` (con `StrictModes yes`) **ignora** el archivo si él o la carpeta `.ssh` tienen permisos de escritura para el grupo u otros. Quedó `drwx------` (700) para `.ssh` y `-rw-------` (600) para `authorized_keys`, ambos de `lizeth`.

3. `ssh horizonte` → `lizeth` / `srv-horizonte`. No pidió contraseña: el alias de `~/.ssh/config` pone `HostName 127.0.0.1`, `User lizeth`, `IdentityFile ~/.ssh/id_ed25519` e `IdentitiesOnly yes` (usa solo esa clave y no prueba otras). La huella del servidor se agregó a `known_hosts` con `ssh-keyscan` después de regenerarla, sin aceptar nada a ciegas.

Parte D — Publicar con SSH

`scp comunicado.html horizonte:/srv/horizonte/www/` → el archivo quedó de **`lizeth`** y del grupo **`webadmin`**.

- **Dueño `lizeth`**: `scp` entra como la usuaria del alias, así que ella crea el archivo.
- **Grupo `webadmin`**: en el Lab 2 la carpeta `/srv/horizonte/www` quedó `root:webadmin` con permisos `2775`. El bit **setgid** (la `s` de `drwxrwsr-x`) hace que todo lo que se crea dentro herede el grupo de la carpeta, y así cualquier miembro de `webadmin` puede editar el sitio.

Primero quedó `-rwxr-xr-x`, porque el archivo vino de Windows con permisos 755 y `scp` los conserva. Se corrigió con `chmod 664` y `scp -p`, y quedó `-rw-rw-r-- lizeth webadmin`, igual que las demás páginas.

En el navegador de Windows, `https://www.horizonte.internal:8443/comunicado.html` muestra **"Comunicado 014/2026 — Publicado por el equipo de TI mediante SSH (scp) con clave ed25519"**. Dentro del servidor, nginx lo sirve: `curl -sk --resolve www.horizonte.internal:443:127.0.0.1 https://www.horizonte.internal/comunicado.html` devuelve el HTML. Con `https://localhost` la conexión se rechaza (código `000`), porque desde el Lab 5 nginx solo responde al nombre del colegio.

- `sftp horizonte` (modo lote `ls -l /srv/horizonte/www`) listó el sitio, con `comunicado.html` de `lizeth webadmin`.
- `rsync -av comunicado.html horizonte:/srv/horizonte/www/` → `sent 107 bytes received 41 bytes`: también va por SSH. Hubo que instalar `rsync`.

Parte E — Verificación

`bash verificar_lab10.sh lizeth` → **Resultado: 11 de 11 comprobaciones.**

Reflexión

Si un día aparece "REMOTE HOST IDENTIFICATION HAS CHANGED", ¿qué hago?

1. **Detenerme y no escribir la contraseña.** El aviso significa que la clave de host no coincide con la de `known_hosts`: o el servidor cambió legítimamente (se reinstaló o se regeneraron las claves, como hicimos en la Parte B) o alguien está en medio de la conexión.
2. **Verificar la huella por otro canal.** Preguntar al administrador o entrar por la consola física o de la VM y ejecutar `ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub`, y comparar con la huella que muestra el aviso.

3. **Solo si coinciden**, borrar la entrada vieja con `ssh-keygen -R <host>` y volver a conectar, aceptando la huella ya verificada.

Qué NO hago: no borro `known_hosts` entero, no uso `StrictHostKeyChecking=no` ni `UserKnownHostsFile=/dev/null` para "que funcione", y no acepto la nueva huella sin verificarla. Hacer eso desactiva justo la protección contra un ataque de intermediario.

Anexo B — BITACORA_IA.md

Bitácora de uso de IA — Laboratorio 10 (TELNET vs SSH).

Estudiante: Lizeth Rodríguez · Entorno: contenedor `horizonte-servidor-lab02` (`srv-horizonte`, Ubuntu 24.04, OpenSSH 9.6p1), Wireshark en Windows · Fecha: 23–25/10/2026

Reglas que respeté (sección 7 de la guía):

- **Nunca** pegué la clave privada en la IA ni en las capturas: de `id_ed25519` solo se mostró el nombre y los permisos con `ls -l`. Lo único que se compartió son huellas (`SHA256:…`) y la clave pública, que son públicas.
- **No** acepté huellas a ciegas: después de regenerar las claves de host, `known_hosts` se llenó con `ssh-keyscan` contra el propio servidor y la huella se comparó con `ssh-keygen -lf`. Las opciones `StrictHostKeyChecking=no` y `UserKnownHostsFile=/dev/null` solo aparecen en el script del docente (usuaria de prueba) y en una consulta de diagnóstico a `localhost` para ver el `kex`; no quedaron en ninguna configuración.
- TELNET se activó solo para la captura y se **desactivó y desinstaló** al terminar (verificador ✅ puerto 23 cerrado).
- Las pruebas fueron únicamente contra mi servidor del curso, y la contraseña de las capturas es la de la usuaria de prueba `secretaria`.

#	Qué pregunté	Qué respondió la IA	Cómo lo verifiqué	Qué aprendí
1	Revisión de mi `preparar_ssh.sh` con los 4 TODO.	Los comandos estaban bien, pero al pegar se habían borrado `EOF` y `fi`, que cierran el bloque `Host horizonte`; sin ellos el script da error de sintaxis.	Reemplacé el archivo y el script terminó con `Conectado como lizeth … con clave ed25519`.	Un heredoc necesita su `EOF` en la columna 1.
2	Qué significa `root@buildkitsandbox` en la huella del servidor (prompt 2).	La clave se creó al construir la imagen; todos los que usan la imagen comparten la misma clave de host y uno podría hacerse pasar por el servidor de otro. Se corrige con `rm /etc/ssh/ssh_host_*` y `ssh-keygen -A`.	Después de `preparar_ssh.sh`: `root@srv-horizonte` con otra huella (`KueRdm7E…`); verificador ✅ claves de host propias.	Clonar VMs o imágenes también clona la identidad.
3	Por qué en el Explorador no aparecía "Seguir flujo TCP".	Esa opción está dentro de Wireshark, que no estaba instalado (los `.pcap` tenían icono genérico).	Instalé Wireshark y abrí `telnet.pcap`: *Analizar* > *Seguir* > *Flujo TCP*.	Distinguir la herramienta del archivo.
4	Cómo leer el flujo TCP de TELNET y de SSH.	En TELNET, rojo = cliente y azul = servidor, todo en texto. En SSH solo se leen la versión y la lista de algoritmos; lo demás va cifrado.	Vi `login:/secretaria` y `Password:/Circular2026`; en SSH, la búsqueda de `Circular2026` no encontró nada.	Qué ve exactamente un espía en la red.
5	Explicar cada algoritmo negociado en mi `ssh -v` (prompt 1).	`sntrup761x25519-sha512`: kex híbrido post-cuántico (NTRU Prime + X25519); `ssh-ed25519`: identidad del servidor; `chacha20-poly1305`: cifrado autenticado (MAC implícito).	Coincide con la lista del flujo SSH en Wireshark y con el verificador (✅ intercambio híbrido post-cuántico); lo contrasté con las notas de versión de OpenSSH 9.0.	Por qué importa el riesgo de "guardar ahora, descifrar después".

#	Qué pregunté	Qué respondió la IA	Cómo lo verifiqué	Qué aprendí
6	Comparar ed25519, ecdsa y rsa para la clave del cliente.	ed25519 tiene claves cortas (256 bits) y es rápido; RSA necesita 3072 bits o más; ECDSA depende de un buen generador aleatorio al firmar. `-a 64` agrega rondas que protegen la privada si tiene frase.	`ls -l ~/.ssh` → `id_ed25519` 600 y `.pub` 644; `authorized_keys` de lizeth con 600.	En la PC real la clave debe tener frase.
7	Por qué `comunicado.html` quedó `-rwxr-xr-x`.	Vino de Windows con 755 por `docker cp` y `scp` conserva los permisos del original.	`chmod 664` + `scp -p` → `-rw-rw-r-- lizeth webadmin`.	Revisar los permisos, no solo dueño y grupo.
8	Por qué `curl https://localhost/comunicado.html` daba `000`.	nginx (Lab 5) solo responde HTTPS al nombre del colegio; hay que pedir `www.horizonte.internal`.	`curl -sk --resolve www.horizonte.internal:443:127.0.0.1 …` devolvió el HTML; `systemctl is-active nginx` → active.	El nombre (SNI) importa en HTTPS.
9	Pasos seguros ante "REMOTE HOST IDENTIFICATION HAS CHANGED" (prompt 3).	Detenerse, verificar la huella por otro canal (consola), comparar y solo entonces `ssh-keygen -R`; nunca desactivar la comprobación.	Lo relacioné con lo que hice: regeneré la clave, `ssh-keygen -R 127.0.0.1` y `ssh-keyscan`, comparando la huella con `ssh-keygen -lf`.	La advertencia es una protección, no un error que hay que callar.

Conclusión

La prueba del espía me mostró con mis propias capturas por qué TELNET no se puede usar: usuario, contraseña y todo lo que se escribe viajan legibles, mientras que en SSH solo se ve la versión. La IA me ayudó a interpretar los algoritmos y a encontrar tres detalles: el `EOF` borrado, Wireshark que faltaba y los permisos del comunicado. Cada respuesta la comprobé con `ls`, `ssh -v`, Wireshark o el verificador. La clave privada nunca salió del servidor y TELNET quedó desinstalado. Resultado: **11 de 11**.